modulo 1

GIT: HERRAMIENTA PARA MANEJAR MEJOR NUESTROS PROYECTOS ORDENADOS

DOS FORMAS DE USARSE

- INTERFAZ GRÁFICA: SE BASA EN VENTANAS (RATÓN Y TECLADO)

COMANDOS → INTERFAZ TEXTO
↓
TERMINAL

- INTERFAZ TEXTO: TE COMUNICAS USANDO SOLO EL TECLADO ↓
SE BASA EN TERMINALES (CASI TODA) MUESTRAN INF ENTRADA E INF SALIDA

- SI EN EL BUSCADOR PONES TERMINAL APARECE LA PANTALLA NEGRA, ES EL SERVIDOR

- UN COMANDO ES UNA INSTRUCCIÓN

comandos

- COMANDO: LO QUE QUEREMOS HACER - ABRIR UN ARCHIVO
- ARGUMENTO: SOBRE LO QUE QUIERO ABRIR - DE HISTORIA
- OPCIONES: PRECEDIDO POR MENOS - TODO EN MAYÚSCULAS

... PUEDE HABER UNO O MÁS

[ ] PUEDE HABER CERO O UNO → ES OPCIONAL

< > TIENE QUE SER AÑADIDO POR EL USUARIO

| PUEDE INCLUIR UNA O A MAS OPCIONES

- ESPACIO: ES LO QUE SE PONE ENTRE COMANDO Y ARGUMENTO

- ls : VER ARCHIVOS
- mkdir : CREAR UNA CARPETA
- cd : PARA METERME DENTRO DE LA CARPETA PARA AÑADIR UN FICHERO
- touch : PARA CREAR EL FICHERO
- nano : PARA MODIFICAR EL FICHERO
  - GUARDAR : control + O → INTRO
  - SALIR : control + X
- pwd : PARA QUE VEAS LA RUTA DONDE SE CREÓ
- rm -r (nombre de la carpeta) : BORRAR LA CARPETA
- rm (nombre del fichero) : BORRAR EL FICHERO
- cat : IMPRIME EN PANTALLA LO QUE CONTIENE EL FICHERO
- cd .. : PARA SALIR DE UNA CARPETA

- DESPUES DE LANZAR UN COMANDO DE GIT, LANZAMOS UN git status
- SI QUERIAMOS EDITAR DOS ARCHIVOS DENTRO DE UNA CARPETA PONEMOS git add. (LO QUE HACE ES COGER TODA LA CARPETA) AÑADIMOS EL COMMIT (LO HACE DE TODO LO QUE HAY DENTRO)

  - PRIMERO - CREAMOS UNA CARPETA
  - SEGUNDO - CREAMOS UN FICHERO
  - TERCERO - GIT INIT (CREAMOS UN REPOSITORIO)
  - CUARTO - GIT STATUTS

- YA EMPEZAMOS A MODIFICAR POR NANO O AÑADIMOS COMMIT
- SE GUARDA SOLO SI AÑADIMOS POR git add, Y YA LE AÑADIMOS EL COMMIT
- git status - PARA VER SI SE AÑADIÓ
- git log - PARA VER LO QUE HEMOS CREADO
- git add - GUARDA LA DIFERENCIA DE LO QUE HABÍA A LO QUE HAY

Scanned with CamScanner

- git mv - RENOMBRAR (MUEVE DIRECTAMENTE AL ÁREA DE STAGING)

- CAMBIAR NOMBRE DEL COMMIT - git commit -m "NOMBRE DEL COMMIT"
- ARREGLAR UN COMMIT - git commit --amend (PARA ARREGLAR CAMBIOS)
  AÑADIR FICHEROS, MODIFICAR PERO SOLO DEL ÚLTIMO COMMIT

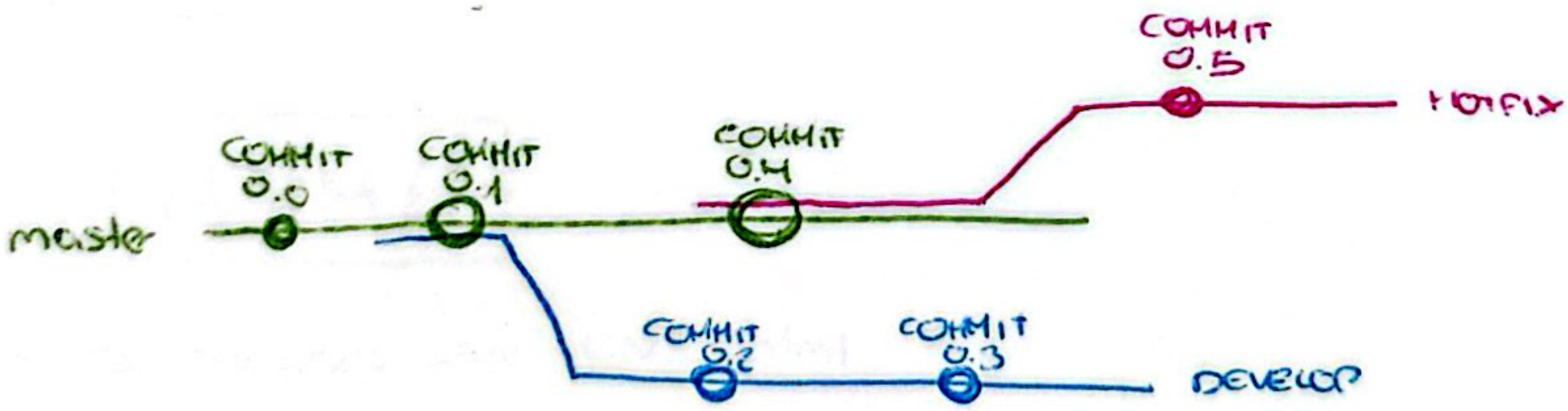
- git log --oneline - MUESTRA LA INFORMACIÓN EN PEQUEÑITO

- HAPPY PATH: COMO LA RUTA QUE ESPERAMOS QUE HAGA EL USUARIO

Importante

- SIEMPRE QUE QUIERAS AÑADIR UN COMMIT TIENES QUE MODIFICARLO CON MANO
- EN UN REPOSITORIO EN EL PRIMER FICHERO SIEMPRE TIENE QUE HABER UN COMMIT
- cp: COPIA UN FICHERO
- git commit: SE PONE DESPUES DE MODIFICAR FICHEROS, Y AÑADIRLOS POR git add Y DESPUES HACEMOS EL COMMIT DE TODO LO QUE TENEMOS ARRIBA

- HASH: ES UNA ESPECIE DE ENCRIPTADO PERO SIN PODER VOLVER ATRÁS, ES LA FORMA DE HACER CLAVES ÚNICAS



- git branch -m : CAMBIA EL NOMBRE DE LA RAMA MASTER
- git switch -c : CREO LA RAMA MASTER Y ENTRO
- CREAMOS UN FICHERO CON TOUCH Y CREAMOS EL COMMIT 0.0 y 0.1
- CREAMOS RAMA DEVELOP CON git branch develop y ENTRAMOS EN LA RAMA CON git switch develop Y CREAMOS COMMIT 0.2 y 0.3
- VOLVEMOS A LA LINEA MASTER CON git switch master y CREAMOS EL

Scanned with CamScanner

COMMIT 0.4.

- CREAMOS LA RAMA HOTFIX CON git switch -c hotfix (YA ESTAMOS EN LA RAMA) Y CREAMOS EL COMMIT 0.5.

- git merge SE HACE EN EL MOMENTO QUE LA RAMA SE SEPARA, ES DECIR, NOS PONEMOS EN LA RAMA DONDE LO QUEREMOS UNIR Y PONEMOS git merge Y EL NOMBRE DE LA RAMA QUE QUEREMOS TRAERNOS

- git log --oneline --all --graph : PARA VERLO COMO UN GRÁFICO

- --no - ff PARA UNIR LAS RAMAS Y NO SE PIERDA INFORMACIÓN, ES DECIR, QUITAMOS LA PARTE RÁPIDA Y SE HACE EL PROCESO PASO A PASO

Scanned with CamScanner